Technical Documentation for CIFAR-10 Experiment

1 Initial and Distilled Models

This section describes the implementation, architecture, training process, and evaluation of both the initial and distilled models for CIFAR-10 classification.

1.1 Model Architecture

The architecture of both the initial and distilled models consists of a series of convolutional and fully connected layers, as summarized in Table 1. The design includes dropout layers for regularization and ReLU activation functions for non-linearity [1].

Table 1: Overview of CIFAR-10 Model Architecture

Layer Type	**CIFAR-10 Architecture**
ReLU Convolutional	64 filters (3x3)
ReLU Convolutional	64 filters (3x3)
Max Pooling	2x2
ReLU Convolutional	128 filters (3x3)
ReLU Convolutional	128 filters (3x3)
Max Pooling	2x2
ReLU Fully Connect.	256 units
ReLU Fully Connect.	256 units
Logits	10 units

1.2 Training Parameters

The training parameters for the initial and distilled models are summarized in Table 2. Key differences include the learning rate, momentum schedule, and the addition of temperature scaling for the distilled model [2].

Table 2: Training Parameters for Initial and Distilled Models

Parameter	**Initial Model**	**Distilled Model**
Learning Rate	0.01 (decayed by 0.5 every 10 epochs)	0.03 (decayed by 0.95 every 5 epochs)
Momentum	0.9 (decayed by 0.5 every 10 epochs)	0.9 (decayed by 0.8 every 10 epochs)
Dropout Rate	0.5	0.5
Batch Size	128	128
Epochs	50	50
Weight Decay	None	1×10^{-4}
Temperature	None	5
Gradient Clipping	None	1.0

1.3 Soft Label Generation

Soft labels for the distilled dataset were generated using the softmax function with temperature scaling. The initial model produces raw logits as output, which are transformed into probabilities during the distillation process [2]:

$$\hat{y}_i = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_j \exp\left(\frac{z_j}{T}\right)},$$

where z_i represents the logits for class i and $T = 5$ is the temperature.

1.4 Theoretical Foundations and Techniques

To enhance the performance and robustness of the models, several theoretical approaches and techniques were employed:

1. **Weight Initialization:** Xavier initialization was used to ensure balanced variance across layers:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right),$$

   where n_{in} and n_{out} denote the number of input and output units [1].

2. **KL Divergence Loss:** For the distilled model, knowledge transfer is achieved using Kullback-Leibler (KL) divergence [4]:

$$D_{KL}(P \parallel Q) = \sum_i P_i \log \frac{P_i}{Q_i},$$

   where P is the soft target distribution and Q is the student model's output.

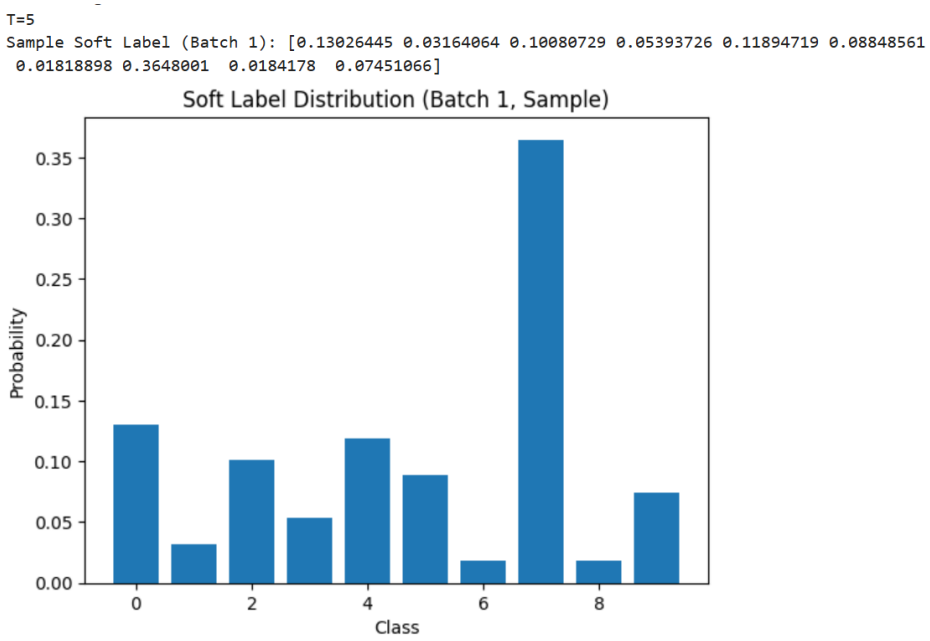


Figure 1: Sample Soft Label Distribution (T=5)

1.5 Accuracy and Loss Analysis

The performance of the initial and distilled models is shown in Figures 2 and 3. These graphs illustrate the differences in test accuracy and training loss over epochs [3].

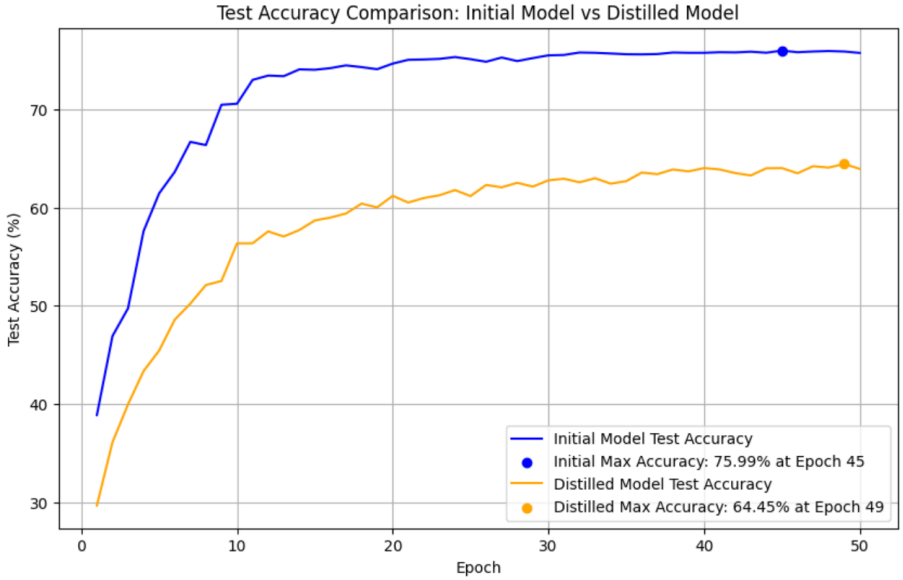


Figure 2: Test Accuracy Comparison: Initial Model vs Distilled Model

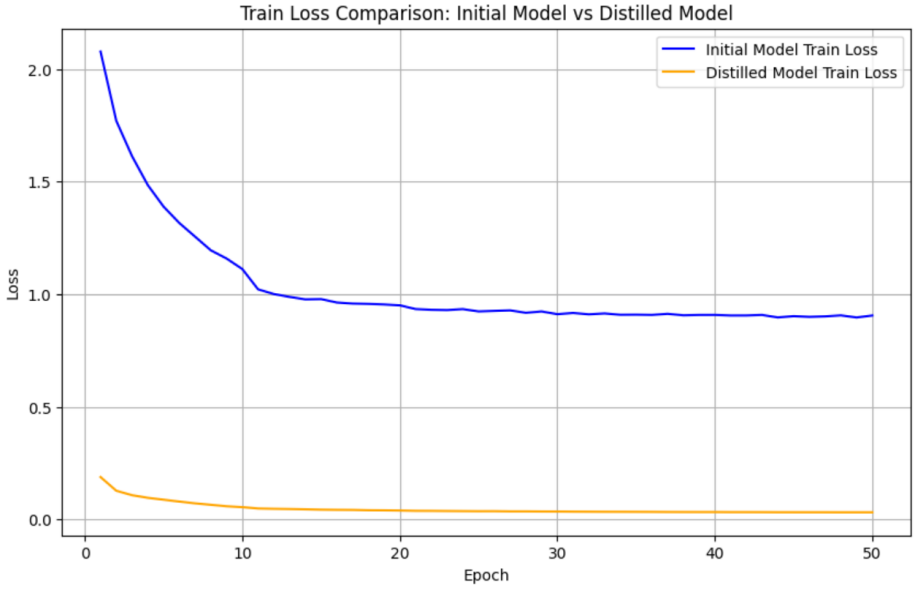


Figure 3: Train Loss Comparison: Initial Model vs Distilled Model

2 Robustness Evaluation

This section evaluates the robustness of the initial and distilled models under adversarial attacks and varying temperature conditions.

2.1 PGD Attack Configuration

The adversarial samples were generated using the Projected Gradient Descent (PGD) method. The PGD algorithm iteratively refines adversarial examples by maximizing the loss function while keeping the perturbations within an ϵ-ball :

$$x_{t+1} = \Pi_{\mathcal{B}(x,\epsilon)} \left(x_t + \alpha \cdot \text{sign} \left(\nabla_x \mathcal{L}(x_t, y) \right) \right),$$

where α is the step size, and $\Pi_{\mathcal{B}}$ projects the adversarial sample back into the ϵ-ball around the original input.

2.2 Theoretical Foundations

Theorem 1 (Lipschitz Continuity [3]). *The robustness of a neural network to small perturbations can be analyzed using its Lipschitz constant K, defined as:*

$$\|f(x_1) - f(x_2)\| \leq K \|x_1 - x_2\|.$$

A smaller K implies a more robust model.

Lemma 1 (Adversarial Robustness Bound [4]). *For any classifier f, the adversarial risk is bounded by:*

$$\mathbb{P}(f(x + \delta) \neq y) \leq \exp\left(-\frac{\|\delta\|^2}{2\sigma^2} \right),$$

where δ is the adversarial perturbation, and σ is the standard deviation of the input noise.

2.3 Attack Success Rates

The attack success rates for both the initial and distilled models were evaluated and are visualized in Figures 4.

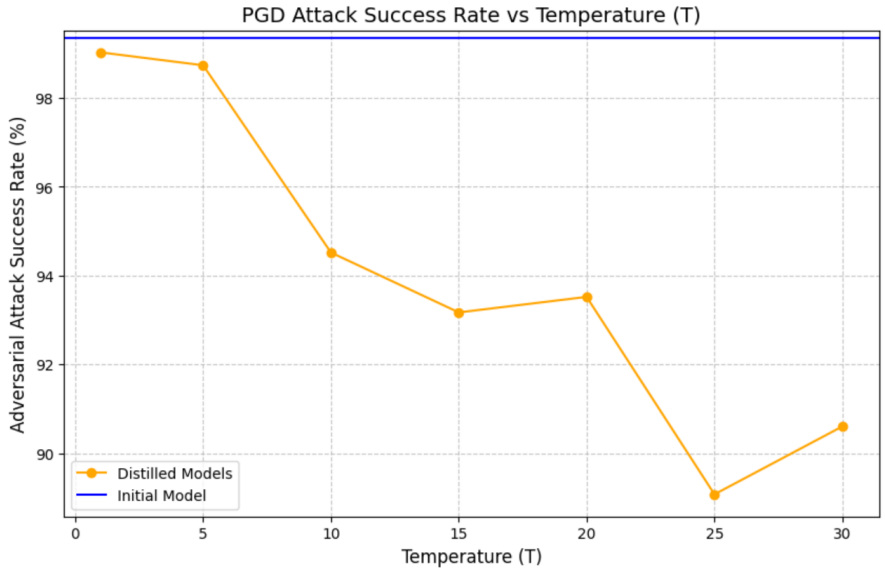


Figure 4: PGD Attack Success Rate

The results indicate that the initial model achieves a PGD attack success rate of **99.36%**. However, the distilled model demonstrates improved robustness, with reduced attack success rates as temperature increases.

3 References

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, 2012, pp. 1097–1105.

[2] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks," *IEEE Symposium on Security and Privacy*, 2016.

[3] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *International Conference on Learning Representations*, 2014.

[4] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *International Conference on Learning Representations*, 2015.